



I need some help with my labsheet where we are using R and Positron.

I'm happy to help with your labsheet using R and Positron! Positron is a free, next-generation data science IDE by Posit that offers first-class support for both R and Python, and it works similarly to RStudio but with some differences.^[1]

To point you in the right direction, could you share more details about what you need help with? For example:

- A specific error or bug you're encountering in your R code
- A concept or function you don't understand (e.g., data wrangling, plotting, statistics)
- Navigating Positron itself (e.g., running scripts, using the console, installing packages)
- Something else from your labsheet

Feel free to paste your code, error messages, or the labsheet instructions, and I'll walk you through it!



To note from here onwards for all code:

You may use third-party packages, including the ggplot2 extension package ggrepel, the scales package for formatting axis breaks and labels, and readxl for importing the World Bank Excel files. Use tidyverse packages for data transformation and analysis unless another package provides specific functionality that substantially simplifies the R code. Every submitted visualization must be produced either by a ggplot() call or by a function wrapper that generates such a plot. Do not submit plots created with base R graphics, lattice, plotly (including plotly::ggplotly()), or other non-ggplot plotting systems. If your team uses a package that is neither a core tidyverse package, ggrepel, scales, nor readxl, briefly explain in a text block before the corresponding code cell what benefit it provides. The core tidyverse packages that you may use without explanation are:

dplyr
forcats
ggplot2
lubridate
purrr
readr
stringr

tibble

tidyr

Below is the first code chunk that I need some help with:

```
#| label: import-data

gdp <-
"API_NY.GDP.PCAP.PP.KD_DS2_en_excel_v2_417139.xls" |>
read_xls(skip = 3) |>
drop_na("2024")

life_exp <-
"API_SP.DYN.LE00.IN_DS2_en_excel_v2_404463.xls" |>
read_xls(skip = 3) |>
drop_na("2024")

pop <-
"API_SP.POP.TOTL_DS2_en_excel_v2_417154.xls" |>
read_xls(skip = 3) |>
drop_na("2024")

gdp
life_exp
pop
```

I am then expected to verify the following from the data:

Confirm that each of the three tibbles contains columns named Country Name, Country Code, and 2024. Additionally, verify that all three tibbles have the same number of rows and that none of the 2024 columns are empty (i.e., they contain at least one non-missing value).

Here's the verification code chunk you should add right after the import block. It checks all the required conditions systematically:

```
#| label: verify-data

# Check column names exist in each tibble
has_required_cols <- function(df, name) {
  required <- c("Country Name", "Country Code", "2024")
  present <- required %in% names(df)
  tibble(
    tibble_name = name,
    column      = required,
    exists      = present
  )
}

bind_rows(
  has_required_cols(gdp,      "gdp"),
  has_required_cols(life_exp, "life_exp"),
  has_required_cols(pop,     "pop")
)
```

```
# Check row counts
tibble(
  tibble_name = c("gdp", "life_exp", "pop"),
  n_rows      = c(nrow(gdp), nrow(life_exp), nrow(pop))
)

# Check that the '2024' column has at least one non-missing value
tibble(
  tibble_name      = c("gdp", "life_exp", "pop"),
  non_na_in_2024 = c(
    sum(!is.na(gdp[["2024"]])),
    sum(!is.na(life_exp[["2024"]])),
    sum(!is.na(pop[["2024"]]))
  )
)
```

What Each Part Does

- `has_required_cols()` — A small helper function that checks whether "Country Name", "Country Code", and "2024" exist in the column names of each tibble, returning a tidy tibble of results. The `%in%` operator returns TRUE/FALSE for each required column name. ^[16]
- **Row count check** — Builds a tibble comparing `nrow()` across all three datasets. If all three tibbles show the same number in the `n_rows` column, the row counts match. ^[16]
- **Non-NA check on 2024** — Uses `sum(!is.na(...))` to count how many non-missing values are in each tibble's 2024 column. A value greater than 0 confirms the column is not entirely empty. Note that `df[["2024"]]` uses double-bracket indexing because the column name starts with a number, which makes `df$2024` invalid syntax in R. ^[16]

Why `drop_na("2024")` Alone Isn't Sufficient

Your import code already uses `drop_na("2024")`, which removes rows where 2024 is NA. However, the verification step above still matters because it independently confirms the column exists and that rows were retained after the drop — catching edge cases like a misnamed column (e.g., "2024 " with a trailing space) that would cause `drop_na()` to silently skip the filter. ^[16]

✱

This is the task that I am struggling with:

Task 3.5) Use a dplyr join function to merge the regions in metadata into countries so that countries still contains the same entries afterward.

This is currently what I have written:

3.3

```
#| label: merged-tibbles
countries <- gdp |>
  left_join(life_exp, join_by(name, code)) |>
  left_join(pop, join_by(name, code))

countries
```

3.4

```
#| label: import-metadata
metadata <-
  "API_NY.GDP.PCAP.PP.KD_DS2_en_excel_v2_417139.xls" |>
  read_xls(sheet = "Metadata - Countries") |>
  select(code = "Country Code", region = "Region")

metadata
```

3.5

```
#| label: join-metadata

countries <- countries |>
left_join(metadata, join_by(code))

countries
```

Please give me suggestions on how I can complete 3.5 and if i am on the right track

You are **on the right track!** A `left_join()` is exactly the correct choice here. Here's why, and what to watch out for:

Why `left_join()` is Correct

The task says *"countries still contains the same entries afterward"* — this is the key constraint. A `left_join()` keeps **all rows from the left table** (`countries`) and brings in matching columns from the right table (`metadata`), filling NA for any countries without a matching region. This guarantees the row count of `countries` is preserved. ^[26]

Using `inner_join()` would drop countries that have no match in `metadata`, and `full_join()` could add extra rows from `metadata` — both would violate the task requirement. ^[26]

Your Code Looks Good — Add the Code Fence

Your logic is correct, you're just missing the opening ````{r}` code fence. Here's the complete, corrected chunk:

```
## 3.5

```{r}
#| label: join-metadata

countries <- countries |>
 left_join(metadata, join_by(code))

countries
```

## One Thing to Verify

After running it, check two things:

1. **Same row count** — `nrow(countries)` should be unchanged from before the join.
2. **region column has some NA's** — This is expected and fine. Some countries in

You can quickly verify with:

```
```r
# Row count unchanged?
nrow(countries)

# How many countries are missing a region?
countries |> filter(is.na(region)) |> select(name, code, region)
```

If you find that many legitimate countries are missing a region, it may be worth checking whether the `code` column in `metadata` matches the `code` column in `countries` exactly (same naming and no whitespace issues).

✱✱

next that I require help with is task 3.7 where I have to do the following:

A subsequent task requires you to add country codes as labels to the plot. To prepare for that task, add to `countries` a `label` column containing the three-letter codes of the following countries:

The 10 countries with the highest GDP per capita

The 10 countries with the lowest GDP per capita

The 10 countries with the longest life expectancy

The 10 countries with the shortest life expectancy

The 10 most populous countries

For all other countries, the label should be an empty string. Note that some countries may belong to several categories.

How do I approach this question?

Great question — the key insight here is to think of it as a **two-step process**: first identify *which* countries need a label, then assign the label (or empty string) using `mutate()` and `if_else()`.

The Core Approach

The task requires you to union five sets of countries (top/bottom 10 by GDP, top/bottom 10 by life expectancy, top 10 by population), then label only those countries with their code, and everyone else with `""`.^[36]

Here's the full solution:

```
#| label: add-labels

labeled_countries <- c(
  countries |> slice_max(gdp,      n = 10) |> pull(code),
  countries |> slice_min(gdp,      n = 10) |> pull(code),
  countries |> slice_max(life_exp, n = 10) |> pull(code),
  countries |> slice_min(life_exp, n = 10) |> pull(code),
  countries |> slice_max(pop,      n = 10) |> pull(code)
)

countries <- countries |>
  mutate(label = if_else(code %in% labeled_countries, code, ""))

countries
```

⚠ Replace `gdp`, `life_exp`, and `pop` with the **actual column names** in your `countries` tibble for those metrics (i.e., the "2024" columns, or whatever names you assigned them during the merge in 3.3).

Why Each Function

- `slice_max()` / `slice_min()` — These dplyr functions efficiently return the top or bottom `n` rows by a given column, without needing to sort manually.^[36]
- `pull(code)` — Extracts the `code` column as a plain character vector, so it can be combined into `labeled_countries`.
- `c(...)` — Concatenates all five vectors into one. Duplicates are fine here — `%in%` handles them automatically.
- `if_else(code %in% labeled_countries, code, "")` — For each row, if the country's code is in the labeled set, use the code as the label; otherwise assign an empty string `""`.^[36]

Handling Ties

If there are ties at position 10 (e.g., two countries share the same GDP), `slice_max()` will by default include all tied rows, potentially giving you more than 10. You can control this with the `with_ties = FALSE` argument if you want strictly 10:

```
countries |> slice_max(gdp, n = 10, with_ties = FALSE) |> pull(code)
```

Check with your labsheet instructions whether ties need to be handled strictly.

**

In task 3.6, i used `dropna()` to remove the missing values which gave me 188.

Now im still on 3.7, I am getting this error when using the method which you have suggested:

Error in `slice_max()`:

! Can't compute indices.

Caused by error:

! `order_by` must have size 188, not size 266.

The error message is the key clue: `order_by` must have size 188, not size 266. This means the column name you're passing to `slice_max()` / `slice_min()` **doesn't exist** in `countries`, so `dplyr` is accidentally picking up a different object (of size 266) from your environment instead. [\[47\]](#)

What's Causing This

In task 3.3, your merge used generic column names like `name` and `code`. But the actual **metric columns** in `countries` are still named "2024" (from the original World Bank Excel files) — and since you joined three tibbles that all have a "2024" column, `dplyr` would have auto-suffixed them to something like `2024.x`, `2024.y`, and `2024` (or similar).

Fix: Check Your Actual Column Names First

Run this to see what columns you actually have:

```
names(countries)
```

You'll likely see something like `2024.x`, `2024.y`, `2024` for GDP, life expectancy, and population respectively. Then **rename them** in task 3.3 or right after so they're usable:

```
# Add this to the end of your 3.3 chunk, or in a new chunk before 3.7
countries <- countries |>
  rename(
    gdp      = `2024.x`,
    life_exp = `2024.y`,
```

```
pop      = `2024`  
)
```

Note: The backticks around ``2024.x`` are required because column names starting with a number need to be escaped in R.

Then Your 3.7 Code Will Work

Once the columns are properly renamed, the `slice_max()` / `slice_min()` calls from before will work correctly:

```
#| label: add-labels  
  
labeled_countries <- c(  
  countries |> slice_max(gdp,      n = 10, with_ties = FALSE) |> pull(code),  
  countries |> slice_min(gdp,      n = 10, with_ties = FALSE) |> pull(code),  
  countries |> slice_max(life_exp, n = 10, with_ties = FALSE) |> pull(code),  
  countries |> slice_min(life_exp, n = 10, with_ties = FALSE) |> pull(code),  
  countries |> slice_max(pop,      n = 10, with_ties = FALSE) |> pull(code)  
)  
  
countries <- countries |>  
  mutate(label = if_else(code %in% labeled_countries, code, ""))  
  
countries
```

The `with_ties = FALSE` is also added here as a safeguard — without it, `slice_max()` / `slice_min()` can error or return unexpected results when the `order_by` column contains NA values.^[48]

✱

In the next section, I am trying to complete task 4.3 where we need to use `geom_text_repel()`. Could you tell me what `geom_text_repel()` is for and how to use it?

`geom_text_repel()` is a function from the `ggrepel` package that adds text labels to a `ggplot2` scatter plot while automatically pushing labels away from each other and away from data points to prevent overlapping. It's the ideal tool for your task since your plot will have up to ~50 labeled countries among 188 points — plain `geom_text()` would result in many unreadable overlapping labels.^[57] ^[58]

Basic Usage

The key aesthetic is `label`, which maps to the column you created in 3.7:

```
library(ggrepel)
```



```
ggplot(countries, aes(x = gdp, y = life_exp)) +
  geom_point(aes(size = pop)) +
  geom_text_repel(aes(label = label))
```

Since your `label` column already contains "" for unlabeled countries, `geom_text_repel()` will naturally skip rendering those — empty strings produce no visible text. ^[59]

Useful Arguments to Know

Argument	What it does
<code>size</code>	Font size of labels
<code>max.overlaps</code>	Max number of overlaps allowed before a label is hidden (default: 10) ^[59]
<code>box.padding</code>	Space between label text and the point
<code>nudge_x / nudge_y</code>	Nudge starting position of labels in a direction
<code>min.segment.length</code>	Controls when a line segment is drawn from label to point

Tip for Your Plot

Since you only want labels for the ~50 selected countries (and "" for everyone else), you can also explicitly filter to avoid `geom_text_repel()` wasting computation on blank labels:

```
ggplot(countries, aes(x = gdp, y = life_exp)) +
  geom_point(aes(size = pop)) +
  geom_text_repel(
    data = countries |> filter(label != ""),
    aes(label = label),
    size = 3,
    max.overlaps = Inf # show all labels, never hide due to overlaps
  )
```

Using `data =` inside `geom_text_repel()` to pass only the labeled subset is a clean pattern — it keeps the `geom_point()` layer showing all 188 countries while only labeling the ones you care about. ^[59]

✱

Am I on the right track with the following:

```
{r}
#| label: add-labels
#| fig-width: 10
#| fig-height: 6
p <- p +
  geom_text_repel(
    aes(label = label),
```

```
show.legend = FALSE,  
)
```

p

Yes, you are on the right track! Your code is valid and will work. Here are a few things to note and consider improving: ^[67]

What's Good

- Adding `geom_text_repel()` as a layer to an existing p object using `p <- p + ...` is perfectly fine ggplot2 pattern. ^[67]
- `show.legend = FALSE` is a good call — it prevents a stray "a" text symbol from appearing in the legend, which is the default behaviour when adding a text geom. ^[68]
- `aes(label = label)` correctly maps your prepared label column.

One Potential Issue

Since your `label` column contains "" for most countries, `geom_text_repel()` will still process all 188 rows, which can slow rendering and cause the repulsion algorithm to behave unexpectedly. It's cleaner to pass only labeled rows via the `data` argument: ^[69]

```
#| label: add-labels  
#| fig-width: 10  
#| fig-height: 6  
  
p <- p +  
  geom_text_repel(  
    data = countries |> filter(label != ""),  
    aes(label = label),  
    show.legend = FALSE  
  )  
  
p
```

Optional but Useful Tweaks

If labels are being cut off or hidden, you can force all of them to display with `max.overlaps = Inf`: ^[69]

```
geom_text_repel(  
  data = countries |> filter(label != ""),  
  aes(label = label),  
  show.legend = FALSE,  
  size = 3,  
  max.overlaps = Inf  
)
```

This is especially useful here since your plot could have up to ~50 labels, and the default `max.overlaps = 10` may silently suppress some of them. ^[69]

✱✱

“`max.overlaps = Inf`” What does `max_overlaps` do?

`max.overlaps` controls how many overlapping labels `geom_text_repel()` is willing to tolerate before it gives up and hides a label entirely. ^[77]

How It Works

`geom_text_repel()` uses an iterative algorithm to push labels away from each other and from data points. When the plot is very crowded, it may be impossible to find a position for every label that doesn't overlap with anything. In those cases, `ggrepel` silently drops the label rather than drawing it overlapping. ^[78]

- **Default:** `max.overlaps = 10` — A label is hidden if it would overlap with more than 10 other labels/points. You'll see a warning like: `ggrepel: 3 unlabeled data points (too many overlaps)`. ^[78]
- `max.overlaps = Inf` — Forces `ggrepel` to draw all labels regardless of how crowded it gets, never hiding any. ^[77]

When to Use Each

Value	Use when...
<code>max.overlaps = 10</code> (default)	You have many labels and are okay with some being hidden to keep the plot clean
<code>max.overlaps = Inf</code>	You have a fixed set of important labels (like your ~50 countries) and must show all of them

For your task, since the labsheet specifically asks you to label those countries, `max.overlaps = Inf` is the safer choice — otherwise some country codes might silently disappear from the plot without you noticing. ^[78]

✱✱

I am on the next task 4.4 where I have to use `geom_smooth()` to fit a single LOESS curve to all the data points. Could you help me with how this would typically be formatted?

Here's how to use `geom_smooth()` for a LOESS curve, applied to your plot:

Basic Format

```
p <- p +  
  geom_smooth(  
    method = "loess",  
    se = FALSE  
  )  
  
p
```

Key Arguments

Argument	What it does
<code>method = "loess"</code>	Specifies locally weighted regression smoothing ^[87]
<code>se = FALSE</code>	Hides the grey confidence interval ribbon around the curve ^[88]
<code>span</code>	Controls wiggleness — smaller (e.g. 0.3) = more flexible, larger (e.g. 0.75) = smoother ^[89]

Important: Fitting a *Single* Curve

The task says "a single LOESS curve to all the data points." This is the important part — if your plot colours points by region, `ggplot2` will by default fit **one curve per colour group** instead of one overall curve. To override this, set `group = 1` inside the aesthetic: ^[87]

```
p <- p +  
  geom_smooth(  
    aes(group = 1),  
    method = "loess",  
    se = FALSE  
  )
```

Setting `group = 1` tells `ggplot2` to treat all points as belonging to the same group, so only one curve is fitted across the entire dataset. ^[89]

What LOESS Does

LOESS (Locally Weighted Scatterplot Smoothing) works by fitting a small local regression around each point, weighting nearby points more heavily. The result is a flexible, non-linear trend curve that follows the overall shape of the data — useful for visualising the relationship between GDP and life expectancy without assuming a straight line. ^[90]



I tried a similar version to what you suggested but got the following output:

Warning: Using size aesthetic for lines was deprecated in ggplot2 3.4.0.

i Please use `linewidth` instead.

`geom_smooth()` using formula = 'y ~ x'

Warning: The following aesthetics were dropped during statistical transformation: size.

i This can happen when ggplot fails to infer the correct grouping structure in the data.

i Did you forget to specify a `group` aesthetic or to convert a numerical variable into a factor?

You actually have two separate warnings here, each with a different fix:

Warning 1: size deprecated for lines → use `linewidth`

Since ggplot2 3.4.0, `size` only applies to **points**, while lines (including the `geom_smooth()` curve) now use `linewidth` instead. This warning is triggered because your global `aes(size = pop)` for bubble sizing is being inherited by `geom_smooth()`, which doesn't understand `size` for a line. ^[97]

Fix: Scope `size = pop` only to `geom_point()`, not globally in `ggplot()`:

```
# Instead of this (size inherited by all layers):
ggplot(countries, aes(x = gdp, y = life_exp, size = pop)) +
  geom_point() +
  geom_smooth(...)

# Do this (size scoped only to geom_point):
ggplot(countries, aes(x = gdp, y = life_exp)) +
  geom_point(aes(size = pop)) +
  geom_smooth(...)
```

Warning 2: Aesthetics dropped / wrong grouping

This is the second warning — `geom_smooth()` is trying to use the `size` aesthetic (which varies per country) to determine groups, resulting in one curve per country instead of one overall curve. This goes away automatically once you fix Warning 1 by scoping `size` to `geom_point()` only. ^[98]

If you still see it after that, add `group = 1` explicitly to `geom_smooth()` as a safeguard:

```
geom_smooth(
  aes(group = 1),
  method = "loess",
  se = FALSE
)
```

Summary of the Full Fixed Structure

```
ggplot(countries, aes(x = gdp, y = life_exp)) +
  geom_point(aes(size = pop, colour = region)) +
  geom_smooth(
    aes(group = 1),
    method = "loess",
    se = FALSE
  ) +
  geom_text_repel(
    data = countries |> filter(label != ""),
    aes(label = label),
    show.legend = FALSE
  )
```

✱✱

Could you tell me how labs() are used to add meaningful axis labels?

`labs()` is a single `ggplot2` function that lets you set labels for **every** part of your plot in one place — axes, title, subtitle, caption, and even legend titles. ^[107]

Basic Format

```
p <- p +
  labs(
    x      = "GDP per Capita (PPP, constant 2017 USD)",
    y      = "Life Expectancy at Birth (years)",
    title  = "GDP per Capita vs. Life Expectancy (2024)",
    subtitle = "Bubble size represents population; colour represents region",
    caption = "Source: World Bank",
    size   = "Population",
    colour = "Region"
  )
```

What Each Argument Does

Argument	What it labels
x	X-axis title
y	Y-axis title

Argument	What it labels
<code>title</code>	Main plot title (top-left)
<code>subtitle</code>	Secondary title shown below the main title ^[108]
<code>caption</code>	Small note at the bottom-right, typically for data sources ^[107]
<code>size</code>	Legend title for the <code>size</code> aesthetic (your bubble sizes)
<code>colour</code>	Legend title for the <code>colour</code> aesthetic (your regions)

Key Points

- **Legend labels** are controlled the same way — use the aesthetic name (e.g., `size`, `colour`) as the argument name. This is especially useful for your plot since `size = pop` and `colour = region` will default to the raw column names `pop` and `region` without `labs()`. ^[109]
- Setting any argument to `NULL` removes that label entirely, e.g. `labs(caption = NULL)`. ^[107]
- `labs()` is the recommended approach — the older `xlab()`, `ylab()`, and `ggtitle()` functions are now superseded. ^[107]

✱✱

For the next task 4.6:

GDP per capita is right-skewed, so a logarithmic x-axis spreads the points more evenly (textbook §11.1.4). Map the x-axis to a base-10 logarithmic scale using `scale_x_log10()`.

To reinforce the chapter’s focus on formatting breaks and labels (textbook §11.1.3):

Set the breaks with `breaks_log()` from the `scales` package so that they fall on reader-friendly values like those in Figure 1.3, and set `minor_breaks` as needed.

Format the tick-mark labels with the `scales` helper `label_comma()` so that a break reads “1,000” rather than the default scientific notation “1e+03”.

Discuss (at most four sentences): The `scales` package also offers `label_currency()`, which would print “\$1,000”. Would that be a better choice than `label_comma()`?

This is the approach that I have taken thus far. Am I on the right track? How do I approach this question:

```
p <- p +
scale_x_log10(
breaks = breaks_log(n = 6),
labels = label_comma()
)
```

p

You are on the right track! Your code is mostly correct — here's a complete breakdown of what to refine and how to address the discussion question.

Your Code is Good — Add `minor_breaks`

The task specifically asks you to set `minor_breaks` as needed. On a log scale, minor breaks are typically placed at intermediate values between the major powers of 10 (e.g., 2,000, 3,000, ... between 1,000 and 10,000). Use `breaks_log()` for minor breaks too, with a higher `n`:^[117]

```
p <- p +
  scale_x_log10(
    breaks      = breaks_log(n = 6) ,
    minor_breaks = breaks_log(n = 10) ,
    labels      = label_comma()
  )

p
```

The higher `n` in `minor_breaks` places more tick marks between major breaks, giving readers better visual reference points on the log scale.^[118]

`label_currency()` vs `label_comma()` — Discussion Answer

`label_currency()` would print labels like \$1,000, implying the values are in a specific currency (US dollars). This would be **misleading** here because World Bank GDP per capita figures are expressed in PPP-adjusted constant international dollars, not nominal USD — attaching a \$ sign suggests a precision and currency specificity the data does not actually have. `label_comma()` is the better choice as it formats numbers readably (e.g., 1,000) without implying a specific currency denomination. The unit context is better conveyed through the axis label in `labs()` rather than baked into the tick labels themselves.^{[119] [120]}

Summary of All Arguments

Argument	Purpose
<code>breaks = breaks_log(n = 6)</code>	Places ~6 major tick marks at reader-friendly log-spaced values ^[118]
<code>minor_breaks = breaks_log(n = 10)</code>	Adds intermediate tick marks for visual guidance ^[117]
<code>labels = label_comma()</code>	Formats ticks as 1,000 instead of 1e+03 ^[119]

✱

how many minor breaks would you suggest?

For a log10 scale covering GDP per capita data (typically ranging from roughly \$1,000 to \$100,000), the conventional approach in data visualisation is to place **9 minor breaks per decade** — at the 2×, 3×, 4×, ... 9× positions between each power of 10.^[142]

Recommended Approach

The cleanest way to achieve this with the `scales` package is:

```
scale_x_log10(  
  breaks      = breaks_log(n = 6),  
  minor_breaks = breaks_log(n = 20),  
  labels      = label_comma()  
)
```

The `n` in `breaks_log()` is a **suggested** count, not an exact one — the function picks the nearest "nice" values. Setting `n = 20` for minor breaks gives you roughly 9 minor ticks per decade, which matches the standard convention for log scales. ^[142]

Why 9 Per Decade?

A log10 decade (e.g., 1,000 → 10,000) has natural subdivisions at 2,000; 3,000; 4,000; 5,000; 6,000; 7,000; 8,000; and 9,000 — that's exactly 8 intermediate values (9 minor breaks counting both endpoints), which is why 9-per-decade is the standard. These give readers meaningful reference points without cluttering the axis. ^[142]

If you find the minor gridlines too visually heavy after rendering, you can reduce their visual weight in `theme()`:

```
theme(panel.grid.minor = element_line(linewidth = 0.2, colour = "grey90"))
```



How do I use `label_currency()` and what is its difference with `label_comma()`?

Here's a clear breakdown of both functions and their differences:

```
label_comma()
```

Formats numbers with comma thousand-separators and nothing else: ^[152]

```
label_comma()(1000)    # → "1,000"  
label_comma()(50000)   # → "50,000"
```

Usage in your plot:

```
scale_x_log10(  
  breaks = breaks_log(n = 6),  
  labels = label_comma()  
)
```

`label_currency()`

Does everything `label_comma()` does, **plus** prepends a currency symbol: ^[153]

```
label_currency()(1000)           # → "$1,000"
label_currency(prefix = "€")(1000) # → "€1,000"
label_currency(prefix = "S$")(1000) # → "S$1,000"
```

Usage:

```
scale_x_log10(
  breaks = breaks_log(n = 6),
  labels = label_currency(prefix = "$")
)
```

Side-by-Side Difference

Feature	<code>label_comma()</code>	<code>label_currency()</code>
Comma separators	✓	✓
Currency symbol	✗	✓ (default \$)
Customisable prefix	✗	✓ (e.g. "€", "S\$")
Example output	50,000	\$50,000

Which to Use for Your Task

`label_comma()` is the better choice here. The World Bank GDP per capita values are in **PPP-adjusted constant 2017 international dollars** — a synthetic unit used for cross-country comparison, not a real spendable currency. Attaching a \$ symbol via `label_currency()` would mislead readers into thinking the values are nominal US dollars, which they are not. The currency context is better communicated through the axis label in `labs()` than through the tick marks themselves. ^[154]

✱✱

does the PPP convert all to the same currency?

Yes, PPP does convert all countries to a common unit — but it's **not the same** as a real currency like the US dollar. Here's the key distinction:

What PPP Does

PPP (Purchasing Power Parity) converts each country's GDP from its local currency into a common unit called the **"international dollar"**. This conversion uses PPP rates — essentially price ratios based on a standardised basket of goods — rather than market exchange rates. ^[162]
^[163]

Why It's Not the Same as USD

An **international dollar** is a *hypothetical* unit defined to have the same purchasing power over GDP as one US dollar has within the United States. So while the numbers *look* like US dollars, they are not actual USD — they represent **what that money could buy domestically**, adjusted for local price levels. ^[162]

For example, if India's GDP per capita (PPP) is \$7,000, it doesn't mean the average Indian earns \$7,000 in USD. It means their income can buy the equivalent of what \$7,000 USD would buy in the United States. ^[164]

Why This Matters for Your Plot

This is precisely why `label_currency()` (which prints \$1,000) is misleading for your axis:

- The \$ symbol implies **nominal US dollars**, which are actual tradeable currency ^[165]
- The World Bank values are in **constant 2017 international dollars** — a synthetic comparison unit that adjusts for both exchange rates *and* local price differences ^[162]
- Using `label_comma()` and clarifying the unit in `labs(x = "GDP per Capita (PPP, constant 2017 international $)")` is the honest and accurate approach

✱✱

Why would `breaks_log(n=6)` when dealing with thousand values like GDP?

Great question! The `n` argument in `breaks_log()` is a **suggestion**, not an exact count — the function internally picks the nearest "nice" round numbers and will produce reader-friendly values like powers of 10.

What `breaks_log(n = 6)` Produces for GDP Data

For a typical GDP per capita range of roughly \$500 to \$150,000, `breaks_log(n = 6)` would place major breaks at values like:

```
1,000 → 3,000 → 10,000 → 30,000 → 100,000
```

These are "nice" log-spaced values that feel natural to a reader — they fall near the 10× and 3× multiples that are conventional for log scales.

Why `n = 6` Specifically?

- `n` is a **target hint**, not a guarantee. `breaks_log()` uses it to determine roughly how spread out the breaks should be, then snaps to the nearest human-readable values.
- For a 3-decade range (e.g., \$500 to \$150,000 spans roughly 2.5 decades), requesting `n = 6` gives you approximately **2–3 breaks per decade**, which is the right density — enough to guide the eye without cluttering the axis.

- If you used `n = 10` for major breaks, you'd get too many ticks crowding the axis. If you used `n = 3`, you'd get too few to be useful.

Quick Rule of Thumb

Data range	Suggested <code>n</code> for major breaks
1 decade (e.g., 1,000–10,000)	<code>n = 4</code> or <code>n = 5</code>
2–3 decades (e.g., 500–150,000)	<code>n = 5</code> or <code>n = 6</code>
4+ decades	<code>n = 6</code> or <code>n = 8</code>

For your GDP data specifically, `n = 6` is a solid choice — you can always tweak it up or down by 1 after rendering and seeing how the plot looks.

✱

How do I use `scale_size_area()`? How can I use it to also set breaks, replace default labels, and adjust `max_size`?

`scale_size_area()` is the recommended scale for bubble plots where size represents a quantity like population. Unlike the default `scale_size()`, it maps values **proportionally to bubble area** (not radius), which means a country with twice the population gets a bubble with twice the area — making visual comparisons accurate. It also ensures a value of 0 maps to a size of 0. ^[182]
[183]

Basic Usage

```
p <- p +
  scale_size_area(max_size = 20)
```

Setting Breaks, Labels, and `max_size`

All three can be set inside the same function call:

```
p <- p +
  scale_size_area(
    max_size = 20,
    breaks   = c(1e6, 10e6, 100e6, 500e6, 1e9),
    labels   = c("1M", "10M", "100M", "500M", "1B"),
    name     = "Population"
  )
```

What Each Argument Does

Argument	What it does
<code>max_size</code>	Sets the diameter (in pt) of the largest bubble — default is 6, which is too small for a bubble chart; values like 15–25 work well ^[182]
<code>breaks</code>	A numeric vector of population values where legend bubbles are drawn — pick round, reader-friendly values ^[184]
<code>labels</code>	A character vector replacing the raw numbers in the legend — same length as <code>breaks</code> ; use readable strings like "1M", "1B" ^[184]
<code>name</code>	The legend title — equivalent to setting it via <code>labs(size = "Population")</code>

Practical Tips

- **Choose breaks that span your data range.** For World Bank population data, values range from ~10,000 (small island nations) to ~1.4 billion (China/India). Good break choices: `c(1e6, 10e6, 100e6, 500e6, 1e9)`.
- `max_size` **affects all bubbles.** Increasing it makes all bubbles larger proportionally, so tweak it after rendering to ensure smaller countries are still visible while large ones don't overlap too much. ^[185]
- **Don't use `scale_size()` for population.** The default `scale_size()` scales by radius, which visually exaggerates differences — `scale_size_area()` is always the right choice when size encodes a quantity. ^[183]



Currently for the ggplot graph, the name of the color is just "region", but I want it to be "Region" instead, how do I change this?

```
basic_plot <- basic_plot +  
  guides(  
    color = guide_legend(  
      order = 1,  
      override.aes = list(alpha = 1, size = 4),  
    ),  
    size = guide_legend(  
      order = 2,  
      reverse = TRUE  
    )  
  )
```

There are two easy ways to rename the legend title from "region" to "Region":

Option 1: Use `labs()` (Recommended)

Simply add `colour = "Region"` to your `labs()` call — this is the cleanest approach: ^[197]

```
basic_plot <- basic_plot +  
  labs(  
    colour = "Region",  
    size    = "Population"  
  )
```

Option 2: Use `title` inside `guide_legend()`

You can also set it directly within your existing `guides()` call using the `title` argument: ^[198]

```
basic_plot <- basic_plot +  
  guides(  
    color = guide_legend(  
      title = "Region",  
      order = 1,  
      override.aes = list(alpha = 1, size = 4)  
    ),  
    size = guide_legend(  
      order = 2,  
      reverse = TRUE  
    )  
  )
```

Which to Use?

Option 1 with `labs()` is generally preferred because it keeps all label-related settings in one place, making the code easier to read and maintain. Reserve `guides()` for controlling legend behaviour (ordering, overriding aesthetics, reversing) rather than naming. ^[197]

**

This is the question:

Does reversing the size legend make the population mapping easier to read? Why?

This is my answer:

Reversing the size legend helps to make the population mapping easier to read as `ggplot2` places the smallest values at the top and the largest at the bottom. This goes against the convention where the y-axis increases upward, which makes it slightly counter-intuitive.

Are there any refinements I should make?

Your answer is on the right track and covers the core idea well. Here are a few refinements to make it more precise and complete:

Suggested Refined Answer

By default, ggplot2 orders size legend keys from smallest at the top to largest at the bottom. This is counter-intuitive because it conflicts with the convention that values increase upward — the same convention used on the y-axis and in most ordered lists. Reversing the legend with `reverse = TRUE` places the largest population bubble at the top and the smallest at the bottom, aligning the legend's visual hierarchy with the reader's natural expectation that "more" is "higher". This makes it faster to look up a country's approximate population by matching the bubble size to the legend.

What You Should Add or Adjust

- **Be more explicit about the direction after reversing** — your answer explains why the default is bad, but doesn't state what `reverse = TRUE` actually produces (largest on top). Mentioning this makes the answer complete.
- **The y-axis analogy is good but slightly imprecise** — the more direct comparison is to general reading convention (larger = higher on a vertical scale), not specifically the y-axis, since the legend is separate from the axes.
- **Optionally mention readability speed** — reversed legends let readers scan from "largest" to "smallest" in natural top-down reading order, which is the more common lookup direction (people are usually interested in identifying the most populous countries first).

Your core reasoning is correct — just flesh out both sides (what the problem is *and* what the reversal achieves) for a complete answer.

This is the next question:

Why should the color legend use opaque symbols (`alpha = 1`) although the plotted points are semi-transparent?

This was my answer:

When we plotted the points with `alpha = 0.5` to reduce overplotting, the legend also inherits this trait and can become hard to read. Therefore, we override the legend symbol to make it opaque and easier to read.

Is there any refinements I should make?

Your answer is correct and concise — it captures the key reason well. Here are a few minor refinements to make it more precise and polished:

Suggested Refined Answer

When points are plotted with `alpha = 0.5` to reduce overplotting in the scatter plot, the colour legend symbols inherit the same transparency by default. Semi-transparent legend swatches appear lighter and washed out, making it difficult to distinguish similar colours — particularly for the region colour palette where differences between hues may already be subtle. Overriding with `alpha = 1` in `override.aes` makes the legend symbols fully opaque, ensuring the colours are displayed at full intensity and are easy to identify, without affecting the transparency of the actual plotted points.

What You Should Add or Adjust

- Explain *why* semi-transparency is a problem in the legend specifically — the purpose of `alpha < 1` in the plot is to reveal overlapping points, but in the legend there is only one symbol per category, so there is nothing to "overlap" and no benefit to transparency there. This makes the justification stronger.
- Clarify that `override.aes` only affects the legend, not the plot — your answer implies this but doesn't state it explicitly. Adding this makes it clear that the change is cosmetic and doesn't alter the underlying data visualisation.

Your answer is already well-reasoned — these additions just make the *why* more rigorous for both sides: why transparency is useful in the plot, and why it is actively harmful in the legend.

What are the instances in which moving the x-axis to the top would be justifiable?

Moving the x-axis to the top is a deliberate design choice suited to specific situations. Here are the main justifiable instances:

When a Top X-Axis Makes Sense

- **Horizontal bar charts ordered top-to-bottom** — When bars are ranked from largest to smallest (reading top-down), placing the x-axis at the top immediately orients the reader to the scale before they start reading the bars, rather than making them scroll to the bottom first. ^[207]
- **Tables or heatmaps** — When the plot resembles a table (e.g., a tile/heatmap with categorical axes), a top x-axis mirrors the convention of column headers in a table, making the chart feel more natural to read. ^[208]
- **When the bottom of the plot is cluttered** — If long y-axis tick labels, annotations, or a caption crowd the bottom of the plot, moving the x-axis to the top avoids visual collision and reduces clutter. ^[209]
- **Dual-axis plots** — When a second x-axis is needed on the opposite side to show a related but different scale (e.g., kilometres on top, miles on bottom), placing one axis at the top is the standard convention. ^[210]
- **Small multiples (facets) where the top row needs labelling** — In a faceted layout, the top x-axis can serve double duty as both axis labels and a header for the top row of facets.

When It Is *Not* Justified

For a standard scatter plot like yours (GDP vs. life expectancy), moving the x-axis to the top would be unconventional and confusing — readers expect the x-axis at the bottom by default. The top position is only justified when the **reading direction or layout of the chart** specifically benefits from it.



Why would it worsen things in my instance of GDP/life-expectancy?

This is a good question to think through for your labsheet discussion. Here are the specific reasons why a top x-axis would worsen your GDP vs. life expectancy scatter plot:

Why It Hurts This Specific Plot

- **Reading flow is broken** — In a scatter plot, the reader's eye naturally travels from the axes inward to the data. With the x-axis at the bottom, you read the GDP scale first, then look up to find the corresponding life expectancy. Moving it to the top forces the reader's eye to start at the top, scan down to find a point, then travel back up to read the x-value — an unnatural back-and-forth motion.
- **The LOESS curve reads from bottom-left to top-right** — The positive trend in your Preston curve rises as you move right across the plot. A bottom x-axis anchors the start of this trend visually, making the direction intuitive. With a top axis, the curve appears to "float" with no grounding reference below it.
- **Labels and captions conflict** — Your plot already has `geom_text_repel()` labels pushed toward the bottom of the plot for low-GDP countries, and likely a caption at the bottom. Moving the x-axis to the top would create a crowded top edge competing with the plot title, subtitle, and axis label, while leaving the bottom empty and unbalanced.
- **No ranked structure justifies it** — The top x-axis is justified when there is a natural top-to-bottom reading order (like a ranked bar chart). A scatter plot has no such ordering — countries are distributed freely across the plot area — so there is no directional reading benefit to gain from moving the axis up.



Even for the below labels?

```
basic_plot <- basic_plot + labs(  
  x = "GDP per capita ($ Purchasing Power Parity 2021)",  
  y = "Life Expectancy (years)",  
  title = "Life Expectancy vs. GDP per Capita (2024)",  
  caption = "Source: World Bank (2024)",  
)
```

Yes, even with your `labs()` settings, keeping the x-axis at the **bottom** is still the right choice. Here's why each element reinforces that decision:

How Your Labels Interact with Axis Position

- **title and subtitle sit at the top** — Your plot title "Life Expectancy vs. GDP per Capita (2024)" already occupies the top of the plot. Moving the x-axis label "GDP per capita (\$ Purchasing Power Parity 2021)" to the top as well would crowd both elements together at the top edge, creating visual competition between the title and the axis label.
- **caption anchors the bottom** — Your "Source: World Bank (2024)" caption sits at the bottom-right. With the x-axis at the bottom, this creates a clean bottom zone: axis ticks → axis label → caption, all stacked neatly. Moving the x-axis to the top leaves the bottom with only the caption, which looks orphaned and unbalanced.
- **Your x-axis label is long** — "GDP per capita (\$ Purchasing Power Parity 2021)" is a fairly long string. At the bottom it sits unobtrusively below the ticks. At the top it would visually merge with the plot title, making readers unsure which text is the title and which is the axis label.

One Small Note on Your Labels

Your x-axis label uses \$ which, as discussed earlier, implies nominal USD rather than the PPP international dollar unit. Consider revising to:

```
x = "GDP per Capita (PPP, constant 2021 international $)"
```

This is more technically accurate and consistent with World Bank's own terminology.

Even if you were to consider that the x-axis title is no longer there? For example:

```
guides(x = guide_axis(title = NULL, position = "top"))?
```

That's a thoughtful edge case to consider. Even with the x-axis title removed, moving it to the top is still not justified for your specific plot — here's why:

Why It Still Worsens the Plot

- **Tick labels at the top conflict with the plot title** — Even without an axis title, the tick mark labels (1,000, 10,000, 100,000) would appear directly beneath your plot title and subtitle, creating a cluttered top edge where numbers and text compete for attention.
- **Scatter plots have no top-to-bottom reading convention** — The justification for a top x-axis in ranked bar charts is that your eye enters the chart at the top and reads downward. In your scatter plot, the reader's eye enters at a data point of interest and then looks *outward* to both axes simultaneously to read coordinates. Neither axis has a privileged "enter here" position, so there is no gain from moving the x-axis up.

- **Low-GDP countries sit near the left, not the top** — In a bar chart ordered top-to-bottom, the top x-axis is close to the most important bars (the largest). In your scatter plot, the most label-dense region (low-GDP, low-life-expectancy countries) sits at the **bottom-left** — far from a top x-axis, making it *harder* to read those points' x-values, not easier.
- **Removing the title loses information** — Dropping "GDP per capita (PPP...)" entirely just to justify moving the axis up is a net loss. The axis title carries important context (the unit and PPP adjustment) that the tick labels alone cannot convey.

Bottom Line

The top x-axis position solves a specific problem — proximity of the axis to the data being read first. Your scatter plot doesn't have that problem, so removing the title to make room is sacrificing useful information for no readability gain.

This is my answer, what do you think?

Moving the x-axis to the top would worsen the presentation of the Preston curve. Currently, our title is "Life Expectancy vs. GDP per Capita (2024)", while the x-axis is "GDP per capita (\$ Purchasing Power Parity 2021)". The title in its current state can not also serve as the x-axis title, and completely changes the meaning of the visualization.

Additionally, people typically expect the independent variable (GDP per Capita) to be at the bottom and the dependent variable (Life Expectancy) to start from the bottom-left corner. Placing the x-axis at the top forces the viewer's eyes to constantly track upward to check values, breaking conventional reading habits.

Therefore, we reject this change.

Your answer is solid and well-structured. Here are some refinements to consider:

What's Strong

- The point about the plot title not being able to substitute as an x-axis title is good and specific to your plot.
- Mentioning reader convention (independent variable on x at the bottom) is a valid and well-recognised principle.
- The conclusion is clear and decisive.

What to Refine

First paragraph — The phrase "completely changes the meaning of the visualization" is a bit vague. Be more specific about *how* it changes the meaning:

"Moving the x-axis title to the top would leave readers uncertain whether 'Life Expectancy vs. GDP per Capita (2024)' is the plot title or the axis label, since both would occupy the same top edge."

Second paragraph — "Track upward to check values" is good, but you can strengthen it by noting *which* countries are most affected:

"Countries with low GDP per capita cluster at the bottom-left of the plot — furthest from a top x-axis — making it hardest to read x-values for precisely the points that require the most scrutiny."

Missing point — You don't mention the caption. The bottom of your plot already has "Source: World Bank (2024)", which creates a natural visual anchor. Moving the x-axis to the top leaves the bottom unbalanced with only the caption sitting alone beneath an empty space.

Revised Version

Moving the x-axis to the top would worsen the presentation of the Preston curve. With both the plot title "Life Expectancy vs. GDP per Capita (2024)" and axis tick labels occupying the top edge, readers would struggle to distinguish the title from axis annotations, creating visual clutter. Additionally, convention places the independent variable (GDP per capita) on the bottom x-axis, with the dependent variable (life expectancy) rising upward from the bottom-left — moving the axis breaks this expectation and forces readers to track upward to check x-values. This is particularly problematic for low-GDP countries, which cluster at the bottom-left and would be furthest from the top axis. Therefore, we reject this change.

Write at least four sentences about the data, choosing one surprising or noteworthy feature revealed by the plot. If you refer to specific countries, ensure their codes appear as labels in the plot. Cite a publication from a trustworthy source that addresses whether the feature is indeed plausible.

Here is a well-structured four-sentence analysis that meets all the requirements of the task:

The plot broadly confirms the Preston curve — life expectancy rises sharply with GDP per capita at lower income levels but flattens among wealthy nations — consistent with IZA research showing that GDP per capita alone explains over 64% of global variation in life expectancy across 213 years of data. A noteworthy outlier is RUS (Russia), which sits well below the LOESS curve despite a GDP per capita of roughly \$30,000 PPP, placing it at a life expectancy comparable to countries with far lower incomes — a pattern described by researchers as a "quasi-resource-curse" where oil and gas revenues inflate GDP without translating into broader social welfare. This anomaly is well-documented: a *JAMA* study found that age-adjusted mortality in Russia rose by nearly 33% between 1990 and 1994, with cardiovascular disease, injuries, and alcohol-related causes accounting for the majority of the decline in life expectancy, reflecting deep structural dysfunction rather than poverty. At the opposite end, HKG (Hong Kong) sits conspicuously above the curve with one of the world's highest life expectancies despite a GDP per capita lower than several Western nations, suggesting that

factors such as diet, healthcare access, and social cohesion can allow countries to substantially outperform what their income level alone would predict. ^[227] ^[228] ^[229] ^[230]



I noticed that the text is slightly translucent, why is it so and how do I make it more bolded/more opaque

The translucency of your `geom_text_repel()` labels is almost certainly caused by the `alpha` aesthetic being inherited from the global `ggplot()` mapping — the same `alpha` you set on `geom_point()` to reduce overplotting is being passed down to the text layer as well. ^[243]

The Fix

Explicitly set `alpha = 1` inside `geom_text_repel()` to make the labels fully opaque, and optionally add `fontface = "bold"` to make them bolder:

```
geom_text_repel(
  data = countries |> filter(label != ""),
  aes(label = label),
  alpha = 1,                # fully opaque, overrides any inherited alpha
  fontface = "bold",        # makes text bolder and easier to read
  show.legend = FALSE,
  size = 3,
  max.overlaps = Inf
)
```

Why This Happens

If your global `aes()` or an earlier layer sets `alpha` (e.g., `geom_point(alpha = 0.5)`), `ggplot2` can propagate that value to subsequent layers. `geom_text_repel()` respects the `alpha` aesthetic, so it inherits the semi-transparency unless you explicitly override it. ^[244]

Quick Summary of Text Appearance Arguments

Argument	Effect
<code>alpha = 1</code>	Fully opaque text ^[244]
<code>fontface = "bold"</code>	Bold text
<code>fontface = "bold.italic"</code>	Bold and italic
<code>size</code>	Font size (in pt)
<code>colour = "black"</code>	Forces a specific text colour regardless of inherited colour



This is my answer, what do you think?

From the plot, we are able to observe a "sine-wave" shape, where there is a general trend that a higher GDP per capita leads to a higher life expectancy. However, we observe that Nigeria (NGA) sits drastically below the curve, and has one of the lowest life expectancy despite having a decent GDP per capita.

According to Rédaction Africanews (2025), we are able to relate the low life expectancy due to persistent health system challenges, which includes a high burden of infectious diseases, poor access to quality care, and frequent health worker strikes. Therefore, while Nigeria does have decent GDP per capita, they seem to have limited medical infrastructure to help combat the infectious diseases, leading to a low life expectancy.

Source: <https://www.africanews.com/2025/10/07/nigerias-life-expectancy-ranked-lowest-globally/>

Your answer is well-reasoned and hits the key requirements. Here are some refinements to consider:

What's Strong

- Identifying NGA as a specific outlier below the curve is a good concrete observation.
- Linking the anomaly to health system challenges (infectious disease burden, access to care) is plausible and well-argued.
- You correctly reason that GDP alone doesn't translate to health outcomes without adequate infrastructure.

What to Refine

"Sine-wave" shape is inaccurate — A sine wave implies oscillation up and down. The Preston curve is better described as a **concave, logarithmic curve** — it rises steeply at low GDP levels and flattens at higher income levels. Use more precise language:

"The plot reveals a concave upward trend, where life expectancy increases steeply with GDP per capita at lower income levels but flattens among wealthier nations."

"Decent GDP per capita" is vague — From the plot, NGA appears to sit at roughly \$5,000–\$6,000 PPP. Naming an approximate value makes the observation more credible and specific.

Source credibility concern — Africanews is a media outlet, not an academic or institutional source. For a lab sheet, a more trustworthy citation would be a WHO report, World Bank publication, or peer-reviewed journal. Additionally, the URL date (October 2025) is worth verifying carefully.

The task requires at least four sentences — your answer has four but they are short. Expanding slightly on *why* GDP doesn't convert to life expectancy in Nigeria's case (e.g., oil revenue concentrated among elites, not reinvested in public health) would strengthen it.

Suggested Revised Version

The plot reveals a concave upward trend consistent with the Preston curve — life expectancy rises steeply with GDP per capita at lower income levels but flattens among wealthier nations. A noteworthy outlier is NGA (Nigeria), which sits well below the LOESS curve despite a GDP per capita of approximately \$5,500 PPP, posting a life expectancy comparable to countries with far lower incomes. This is plausible: Nigeria's oil revenues are highly concentrated and have not been broadly reinvested in public health infrastructure, resulting in persistent challenges including a high burden of infectious diseases, poor access to quality care, and frequent health worker strikes. The WHO (2024) notes that Nigeria accounts for a disproportionate share of global maternal and child mortality, confirming that national income alone is insufficient to drive health outcomes without effective public health systems.

This is the question:

Discuss (at most four sentences): The scales package also offers `label_currency()`, which would print “\$1,000”. Would that be a better choice than `label_comma()`?

This is my answer:

b. Using `label_currency()` (which prints \$1,000) would be misleading here because the unit is a "common currency" adjusted for PPP, not nominal US dollars. A reader could incorrectly interpret the values as market-rate USD. `label_comma()` presents the numbers clearly without implying a specific currency, making it the better choice.

What do you think?

Your answer is accurate and well-argued. Here are some refinements:

What's Strong

- Correctly identifying that the values are PPP-adjusted and not nominal USD is the crux of the argument — well done.
- The point about misleading the reader into interpreting values as market-rate USD is precise and directly relevant.
- The conclusion is clear and justified.

What to Refine

"Common currency" is slightly imprecise — The more accurate term is "international dollar", which is the synthetic unit used by the World Bank for PPP-adjusted comparisons. Using the correct terminology strengthens the argument:

"...the unit is the international dollar, a synthetic unit used for cross-country PPP comparison, not nominal US dollars."

You could add one more sentence — The task allows up to four sentences and you only use three. You could briefly acknowledge when `label_currency()` *would* be appropriate, which

shows you understand the function's purpose rather than just rejecting it:

"label_currency() would be appropriate if the data were expressed in a real, tradeable currency such as nominal USD or SGD."

Suggested Revised Version

Using `label_currency()` (which prints \$1,000) would be misleading here because the x-axis values are expressed in international dollars — a synthetic unit used for PPP-adjusted cross-country comparison — not nominal US dollars. A reader could incorrectly interpret the values as market-rate USD, overstating the incomes of lower-income countries whose purchasing power at market exchange rates is significantly lower. `label_comma()` presents the numbers clearly without implying a specific currency denomination, making it the better choice. `label_currency()` would only be appropriate if the data were denominated in a real, tradeable currency such as nominal USD.

This is the question,

With `breaks_log(n = 6)`, `ggplot2` places ticks by default at \$300, \$1,000, \$3,000, \$10,000, \$30,000, and \$100,000 (powers of ten times 1 or 3). In contrast, `Gapminder` chooses ticks at \$1,000, \$2,000, \$4,000, \$8,000, \$16,000, \$32,000, and \$64,000 (successive doublings). Apart from the difference in the number of ticks, state one advantage of each approach for reading GDP per capita off the axis.

This is my answer:

For the `breaks_log(n=6)` approach, the ticks use familiar base-10 increments, making it easy to read off an exact GDP figure because each tick is a simple multiple of a power of ten.

For the successive doublings approach, moving one tick to the right always means the country is exactly twice as wealthy. This makes it immediately intuitive to compare relative income ratios between countries.

What do you think?

Your answer is excellent — both points are accurate, well-phrased, and directly address the question. Here are some minor refinements:

What's Strong

- The base-10 advantage is correctly framed around **reading exact values** — this is the right axis for that argument.
- The doubling advantage is correctly framed around **comparing ratios between countries** — this is precisely the geometric property of equal-ratio spacing.
- Both answers are concise and clearly contrasted.

What to Refine

First point — The phrase "simple multiple of a power of ten" is slightly awkward. Consider tightening it:

"...each major tick falls on a round, familiar value (e.g., 1,000 or 10,000), making it easy to read off an approximate GDP figure without mental arithmetic."

Second point — You could make the comparative advantage even sharper by noting that the doubling scale is less useful for reading *absolute* values (e.g., \$64,000 is less intuitive than \$100,000), which implicitly reinforces why the two approaches serve different purposes:

"...however, the tick values themselves (e.g., \$64,000) are less round and harder to read off precisely than powers of ten."

Suggested Refined Version

For the `breaks_log(n = 6)` approach, ticks fall on round, familiar values such as \$1,000, \$10,000, and \$100,000, making it easy for a reader to estimate a country's GDP per capita at a glance without any mental calculation.

For the successive doubling approach, each step to the right represents exactly a doubling of income, making it immediately intuitive to compare the relative wealth ratio between two countries — for example, a country two ticks to the right is four times wealthier, regardless of where on the axis they sit.

Overall your answer is strong — the refinements are minor and optional. The core reasoning is correct and well-matched to each approach's geometric property.

For the question:

Use `labs()` to add meaningful axis labels, legend labels, and a title. Label the x-axis so that it effectively communicates the units of the GDP indicator. Refer to the Metadata -

Indicators sheet of the World Bank's GDP Excel file and, if necessary, cite trustworthy online sources that explain the purchasing-power adjustment. Add a caption that credits the World Bank as the data source.

This is my answer for it:

4.5 Add Annotations

```
#| label: add-annotation

basic_plot <- basic_plot + labs(
  x = "GDP per capita ($ Purchasing Power Parity 2021)",
  y = "Life Expectancy (years)",
  title = "Life Expectancy vs. GDP per Capita (2024)",
  caption = "Source: World Bank (2024)",
)
```

```
basic_plot
```

According to the World Bank (under Metadata - Indicators), the GDP indicator is adjusted using purchasing power parities (PPPs) and expressed in constant 2021 international dollars. This adjustment controls for differences in local price levels, allowing for an accurate cross-country comparison of true economic output and standard of living.

Am I on the right track?

You are on the right track, but there are a few refinements to make your answer more complete and accurate:

What's Strong

- The code structure is correct — `labs()` is properly used. ^[258]
- The written explanation correctly identifies the key properties: PPP-adjusted, constant international dollars, and cross-country comparability.
- Citing the Metadata - Indicators sheet is exactly what the question asks for.

What to Refine

X-axis label is slightly imprecise — "\$ Purchasing Power Parity 2021" reads awkwardly. The World Bank's own terminology is "constant 2021 international dollars". A cleaner label would be: ^[259]

```
x = "GDP per Capita, PPP (constant 2021 international $)"
```

Missing legend labels — The question explicitly asks for **legend labels** too. Your `labs()` call is missing `colour` and `size` labels: ^[258]

```
colour = "Region",  
size   = "Population"
```

Written explanation could cite a source — The question says "if necessary, cite trustworthy online sources." Since PPP is a technical concept, citing the World Bank DataBank glossary directly strengthens your answer.

Suggested Revised Code

```
#| label: add-annotation  
  
basic_plot <- basic_plot + labs(  
  x      = "GDP per Capita, PPP (constant 2021 international $)",  
  y      = "Life Expectancy at Birth (years)",  
  title  = "Life Expectancy vs. GDP per Capita (2024)",  
  caption = "Source: World Bank (2024)",  
  colour = "Region",
```

```
size      = "Population"
)

basic_plot
```

Suggested Revised Written Explanation

According to the World Bank Metadata - Indicators sheet, the GDP indicator is expressed in constant 2021 international dollars, adjusted using purchasing power parity (PPP) rates. PPP conversion eliminates differences in price levels across countries, so the values reflect what each dollar can actually buy domestically rather than market exchange rates (World Bank DataBank, 2024). This allows for a fair cross-country comparison of living standards, which is why the axis label specifies "international \$" rather than nominal USD.

*
**

1. <https://positron.posit.co/faqs.html>
2. https://cran.r-project.org/web/packages/rUM/vignettes/aa_using_posit_cloud.html
3. <https://www.ixpantia.com/en/blog/positron-from-rstudio>
4. https://www.reddit.com/r/rstats/comments/1ntmik7/have_a_bad_feeling_about_positron/
5. <https://positron.posit.co>
6. <https://www.youtube.com/watch?v=ndq2Mm3Dju8>
7. <https://www.jumpingrivers.com/blog/why-move-to-positron-r/>
8. <https://rstudio.github.io/cheatsheets/positron.pdf>
9. <https://positron.posit.co/install.html>
10. <https://cloud.r-project.org/web/packages/positron.tutorials/positron.tutorials.pdf>
11. <https://devstart.org/CONTENT/GettingStarted/GettingStartedWithR.html>
12. <https://rstudio.github.io/cheatsheets/>
13. <https://posit.co/blog/a-quick-tour-of-positron>
14. <https://cran.r-project.org/web/packages/positron.tutorials/readme/README.html>
15. <https://github.com/posit-dev/positron/discussions/8070>
16. <https://tidyverse.tidyverse.org>
17. <https://nuitrcs.github.io/r-tidyverse/html/others.html>
18. <https://exts.ggplot2.tidyverse.org/ggrepel.html>
19. <https://ggplot2.tidyverse.org>
20. <https://tidyverse.org/packages/>
21. <https://ggplot2.tidyverse.org/reference/index.html>
22. <https://www.r-bloggers.com/2020/06/a-very-short-introduction-to-tidyverse/>
23. <https://friendly.github.io/6135/lectures/ggplot-further.pdf>
24. <https://www.sfu.ca/~mjbrydon/tutorials/BAinR/data.html>

25. <https://manncz.github.io/stat-331-s25/slides/week-2/w2-import-ggplot.html>
26. <https://dplyr.tidyverse.org/reference/mutate-joins.html>
27. <https://www.youtube.com/watch?v=jX6vwg2YK68>
28. https://shiny.abdn.ac.uk/Stats/tutorials/R_Intermediate/R_Intermediate_P1/merge-data-using-dplyr.html
29. <https://codesignal.com/learn/courses/advanced-data-transformation-techniques/lessons/merging-data-frames-in-r-with-dplyr>
30. https://dplyr.tidyverse.org/reference/join_by.html
31. <https://www.codecademy.com/learn/learn-dplyr/modules/learn-r-multiple-tables/cheatsheet>
32. https://coletl.github.io/tidy_intro/lessons/dplyr_join/dplyr_join.html
33. <https://www.infoworld.com/article/2264570/how-to-merge-data-in-r-using-r-merge-dplyr-or-datatable.html>
34. <https://r4ds.hadley.nz/joins.html>
35. <https://www.youtube.com/watch?v=jAl0fRqxK3s>
36. <https://www.codecademy.com/learn/learn-dplyr/modules/learn-r-multiple-tables/cheatsheet>
37. https://en.wikipedia.org/wiki/Country_code
38. https://en.wikipedia.org/wiki/ISO_3166-1
39. https://www.youtube.com/watch?v=ZGD0kO_iwAM
40. <https://www.youtube.com/watch?v=6kB4jqZ1aXM>
41. <https://rforpoliticalscience.com/2020/07/25/include-country-labels-to-a-regression-plot-with-ggplot2-package-in-r/>
42. <https://www.youtube.com/watch?v=2qgn7hpSRDI>
43. <https://www.iso.org/iso-3166-country-codes.html>
44. <https://gist.github.com/stevewithington/20a69c0b6d2ff846ea5d35e5fc47f26c>
45. <https://unstats.un.org/unsd/methodology/m49/>
46. <https://hjalli.medium.com/dealing-with-countries-in-data-ed97ba4cc1a0>
47. <https://dplyr.tidyverse.org/reference/slice.html>
48. <https://github.com/tidyverse/dplyr/issues/4826>
49. <https://r4ds.hadley.nz/missing-values.html>
50. <https://stackoverflow.com/questions/75255909/is-slice-maxn-0-not-possible>
51. <https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.dropna.html>
52. <https://www.geeksforgeeks.org/python/python-pandas-dataframe-dropna/>
53. https://tidyr.tidyverse.org/reference/drop_na.html
54. <https://www.spsanderson.com/steveondata/posts/2024-12-12/>
55. <https://www.youtube.com/watch?v=TkYXEII3aZw>
56. <https://github.com/tidyverse/dplyr/issues/6176>
57. <https://www.youtube.com/watch?v=IEsyqhQRHTs>
58. https://ggrepel.slowkow.com/reference/geom_text_repel.html
59. <https://r-graph-gallery.com/package/ggrepel.html>
60. https://lets-plot.org/python/pages/api/lets_plot.geom_text_repel.html

61. <https://stackoverflow.com/questions/46910696/labeling-with-geom-text-and-geom-text-repel>
62. <https://cran.r-project.org/web/packages/ggrepel/readme/README.html>
63. https://www.rdocumentation.org/packages/ggrepel/versions/0.5/topics/geom_label_repel
64. <https://www.r4photobiology.info/galleries/nudge-and-repel.html>
65. <https://github.com/slowkow/ggrepel/blob/master/vignettes/examples.Rmd>
66. <https://rdr.io/cran/ggrepel/src/R/geom-text-repel.R>
67. <https://cran.r-project.org/web/packages/ggrepel/vignettes/ggrepel.html>
68. <https://r-charts.com/ggplot2/text-annotations/>
69. <https://r-graph-gallery.com/package/ggrepel.html>
70. <https://github.com/slowkow/ggrepel/blob/master/vignettes/examples.Rmd>
71. https://www.rdocumentation.org/packages/ggrepel/versions/0.4/topics/geom_label_repel
72. https://ggplot2.tidyverse.org/reference/geom_text.html
73. <https://cran.r-project.org/web/packages/ggrepel/ggrepel.pdf>
74. <https://github.com/slowkow/ggrepel/blob/master/R/geom-text-repel.R>
75. https://datavizf23.classes.andrewheiss.com/news/2023-11-29_faqs_week-11-12-13.html
76. https://ggrepel.slowkow.com/reference/geom_text_repel.html
77. <https://r-graph-gallery.com/package/ggrepel.html>
78. <https://stackoverflow.com/questions/69922847/warning-message-ggrepel-1-unlabeled-data-points-too-many-overlaps-consider>
79. <https://blog.csdn.net/weshengxin/article/details/129761355>
80. <https://support.bioconductor.org/p/9136394/>
81. <https://cran.r-project.org/web/packages/ggrepel/ggrepel.pdf>
82. <https://ggrepel.slowkow.com/articles/examples>
83. <https://cran.r-project.org/web/packages/ggrepel/news/news.html>
84. <https://rdr.io/cran/ggrepel/f/NEWS.md>
85. <https://www.r4photobiology.info/galleries/nudge-and-repel.html>
86. <https://github.com/slowkow/ggrepel/issues/48>
87. <https://rforpoliticalscience.com/2023/10/09/locally-weighted-scatterplot-smoothing-loess-in-r/>
88. <https://www.datanovia.com/en/blog/how-to-plot-a-smooth-line-using-ggplot2/>
89. https://www.rdocumentation.org/packages/ggplot2/versions/3.3.3/topics/geom_smooth
90. https://www.reddit.com/r/rstats/comments/xppp2s/used_geom_smooththis_the_blue_line_the_mean_of_it/
91. <https://stackoverflow.com/questions/52097776/loess-smoothing-geom-smooth-vs-loess>
92. https://web.math.ku.dk/~richard/courses/compstat2020/S3smoother_static.html
93. https://plotly.com/ggplot2/layers/geoms/geom_smooth/
94. <https://stackoverflow.com/questions/72325105/loess-vs-geom-smooth-giving-different-curves>
95. <https://stackoverflow.com/questions/71781444/fitting-loess-function-in-ggplot>
96. <https://stackoverflow.com/questions/72872124/geom-smooth-function-error-with-loess-method-in-ggplot2>

97. https://ivelasq.github.io/2023-03-22_whats-new-in-the-tidyverse/posts/ggplot2-linewidth/
98. <https://github.com/tidyverse/ggplot2/issues/5185>
99. <https://github.com/tidymodels/infer/issues/477>
100. <https://github.com/wilkelab/ggbridges/issues/78>
101. <https://github.com/tidyverse/ggplot2/issues/5051>
102. <https://stackoverflow.com/questions/74750478/correct-syntax-for-manually-scaling-the-width-of-lines-in-ggplot>
103. <https://onepager.togaware.com/faceted-location-thin-lines.html>
104. <https://github.com/rivasiker/ggHoriPlot/issues/2>
105. <https://tidyverse.org/blog/2022/11/ggplot2-3-4-0/>
106. <https://tidyverse.org/blog/2022/08/ggplot2-3-4-0-size-to-linewidth/>
107. <https://ggplot2.tidyverse.org/reference/labs.html>
108. <https://viz-ggplot2.rsquaredacademy.com/ggplot2-labels>
109. <https://www.datanovia.com/en/blog/how-to-change-ggplot-labels/>
110. <https://www.andrewheiss.com/blog/2022/06/23/long-labels-ggplot/>
111. <https://www.rdocumentation.org/packages/ggplot2/versions/2.1.0/topics/labs>
112. <https://stackoverflow.com/questions/10438752/adding-x-and-y-axis-labels-in-ggplot2>
113. <https://www.datanovia.com/en/blog/ggplot-axis-labels/>
114. <https://posit.cloud/learn/recipes/visualize/VisualizeG>
115. <https://stackoverflow.com/questions/70296079/how-to-modify-axis-labels-within-ggplot-labs>
116. <https://stackoverflow.com/questions/74564625/make-x-axis-labels-value-labels-and-other-axis-labels-bold-in-ggplot2>
117. https://ggplot2.tidyverse.org/reference/scale_continuous.html
118. <https://cran.r-project.org/web/packages/scales/readme/README.html>
119. https://scales.r-lib.org/reference/label_number.html
120. https://rdr.io/cran/scales/man/dollar_format.html
121. <https://github.com/jgscott/learnR/blob/master/infmort/infmort.md>
122. https://en.wikipedia.org/wiki/Logarithmic_scale
123. https://data-vis-using-r.info/11_scales/scales.html
124. <https://r.tiid.org/graphs.html>
125. <https://www.andrewheiss.com/blog/2022/12/08/log10-natural-log-scales-ggplot/>
126. <https://groups.google.com/g/ggplot2/c/aAB2MqfiDz8>
127. <https://r4ds.github.io/bookclub-ggplot2/labels.html>
128. https://rdr.io/github/ConGibbs10/UtilsGibbs/man/log10_minor_breaks.html
129. <https://stackoverflow.com/questions/64922259/add-minor-breaks-to-ggplot-without-labels>
130. <https://datavizf23.classes.andrewheiss.com/example/05-example.html>
131. <https://stackoverflow.com/questions/14255533/pretty-ticks-for-log-normal-scale-using-ggplot2-dynamic-not-manual/22227846>

132. <https://stackoverflow.com/questions/69588182/adding-a-comma-and-a-plus-sign-to-numbers-on-the-x-axis-of-ggplot>

133. https://search.r-project.org/CRAN/refmans/scales/html/minor_breaks_width.html

134. <https://stackoverflow.com/questions/63539225/different-numeric-scale-labels-for-ggplot2>

135. <https://onepager.togaware.com/labels-with-dollars.html>

136. <https://ggplot2-book.org/scales-position.html>

137. <https://campus.datacamp.com/courses/introduction-to-the-tidyverse/data-visualization?ex=5>

138. <https://campus.datacamp.com/courses/introduction-to-the-tidyverse/data-visualization?ex=7>

139. <https://www.oreilly.com/library/view/the-r-book/9780470510247/ch027-sec003.html>

140. https://r-statistics.co/ggplot2-scale_x_log10-in-R.html

141. <https://sciencicity.github.io/scale-ggplot-code-snippet/index.html>

142. <https://groups.google.com/g/ggplot2/c/aAB2MqfiDz8>

143. https://scales.r-lib.org/reference/minor_breaks_width.html

144. <https://time.com/6210430/micro-breaks-work-productivity/>

145. <https://yourstory.com/2025/09/science-perfect-length-breaks>

146. https://www.reddit.com/r/bandmembers/comments/1so8a52/three_hour_set_how_many_breaks_is_appropriate/

147. <https://ehs.stanford.edu/subtopic/microbreaks>

148. <https://stackoverflow.com/questions/6492259/add-minor-breaks-to-ggplot-without-labels>

149. <https://www.lni.wa.gov/workers-rights/youth-employment/wages-rest-breaks-and-meal-periods>

150. <https://time.com/3518053/perfect-break/>

151. <https://www.mom.gov.sg/employment-practices/young-persons-and-children>

152. <https://onepager.togaware.com/labels-with-comma.html>

153. <https://tidyverse.org/blog/2022/04/scales-1-2-0/>

154. https://scales.r-lib.org/reference/label_number.html

155. <https://r4ds.github.io/bookclub-ggplot2/labels.html>

156. <https://masaqol.hatenablog.com/entry/2020/12/31/173744>

157. <https://onepager.togaware.com/labels-with-dollars.html>

158. <https://stackoverflow.com/questions/37713351/formatting-ggplot2-axis-labels-with-commas-and-k-mm-if-i-already-have-a-y-sc/70391643>

159. <https://stackoverflow.com/questions/63539225/different-numeric-scale-labels-for-ggplot2>

160. <https://forum.posit.co/t/ggplot2-histogram-of-frequencies-and-axis-formatting-comma-instead-is-scientific-in-facet-grid/66720>

161. <https://stackoverflow.com/questions/37713351/formatting-ggplot2-axis-labels-with-commas-and-k-mm-if-i-already-have-a-y-sc>

162. <https://databank.worldbank.org/metadataglossary/jobs/series/NY.GDP.PCAP.PP.KD>

163. <https://www.oecd.org/en/data/datasets/purchasing-power-parities.html>

164. <https://www.iosrjournals.org/iosr-jef/papers/Vol14-Issue2/Ser-1/A1402010102.pdf>

165. https://www.reddit.com/r/AskEconomics/comments/t1y55f/whats_the_difference_between_gdp_gdp_per_capita/

166. https://gateway.euro.who.int/en/indicators/hfa_34-0270-real-gross-domestic-product-ppp-per-capita/

167. [https://en.wikipedia.org/wiki/List_of_countries_by_GDP_\(PPP\)_per_capita](https://en.wikipedia.org/wiki/List_of_countries_by_GDP_(PPP)_per_capita)
168. <https://www.youtube.com/watch?v=jIRzzxpksoU>
169. <https://www.investopedia.com/updates/purchasing-power-parity-ppp/>
170. <https://www.oecd.org/en/data/indicators/purchasing-power-parities-ppp.html>
171. <https://data.worldbank.org/indicator/NY.GDP.PCAP.PP.CD>
172. <https://github.com/gshs-ornl/wbstats/blob/master/README.Rmd>
173. <https://api.rogue-scholar.org/posts/10.59350/b4gjd-50c81.pdf>
174. <https://www.statalist.org/forums/forum/general-stata-discussion/general/222486-log-gdp-per-capita-and-log-population-in-the-covariates>
175. https://www.reddit.com/r/academiceconomics/comments/17q7ifz/why_use_log_gdp_instead_of_just_gdp/
176. <https://www.econ.iastate.edu/ask-an-economist/difference-between-gdp-and-log-squared-gdp>
177. <https://www.nature.com/articles/s41597-025-04487-x>
178. <https://databank.worldbank.org/metadataglossary/statistical-capacity-indicators/series/5.51.01.10.gdp>
179. <https://rpubs.com/tbaker262/675370>
180. <https://www.statalist.org/forums/forum/general-stata-discussion/general/1680510-how-can-i-create-a-new-variable-equal-to-the-logarithm-of-gdp-per-capita>
181. <https://www.julianfludwig.com/eda/data-transformation.html>
182. https://www.rdocumentation.org/packages/ggplot2/versions/1.0.1/topics/scale_size_area
183. https://www.rdocumentation.org/packages/ggplot2/versions/2.2.1/topics/scale_size
184. https://search.r-project.org/CRAN/refmans/animint2/html/scale_size.html
185. [https://stackoverflow.com/questions/63023877/setting-absolute-point size for geom-point with scale-size-area](https://stackoverflow.com/questions/63023877/setting-absolute-point-size-for-geom-point-with-scale-size-area)
186. https://lets-plot.org/python/pages/api/lets_plot.scale_size_area.html
187. https://ggplot2tor.com/scales/scale_size_area/
188. <https://stackoverflow.com/questions/29909182/with-scale-size-area-add-lines-from-top-and-bottom-to-next-area-point>
189. <https://bookdown.dongzhuoer.com/hadley/ggplot2-book/size>
190. <https://stackoverflow.com/questions/56098080/force-size-aesthetic-to-scale-to-given-breaks>
191. <https://stackoverflow.com/questions/60441729/scale-size-area-utilizing-max-size-gives-plot-with-points-exceeding-plots-bound>
192. https://plotnine.org/reference/scale_size_area.html
193. <https://rdr.io/cran/ggplot2/src/R/scale-size.R>
194. <https://cloud.tencent.com/developer/ask/sof/354871>
195. <https://www.cnblogs.com/wkslearner/p/5616869.html>
196. <https://stackoverflow.com/questions/37687544/using-scale-size-area-ggplot2-to-plot-points-of-size-0-as-completely-absent>
197. <https://r-graph-gallery.com/239-custom-layout-legend-ggplot2.html>
198. https://ggplot2.tidyverse.org/reference/guide_legend.html
199. <https://ggplot2.tidyverse.org/articles/faq-customising.html>

200. <https://forum.posit.co/t/legend-aes-override-in-ggplot2/188072>

201. <https://ggplot2.tidyverse.org/reference/guides.html>

202. <https://aosmith.rbind.io/2020/07/09/ggplot2-override-aes/>

203. <https://www.datanovia.com/en/blog/ggplot-legend-title-position-and-labels/>

204. <https://ggplot2-book.org/scales-colour.html>

205. <https://github.com/tidyverse/ggplot2/blob/main/R/guide-legend.R>

206. <https://viz-ggplot2.rsquaredacademy.com/ggplot2-modify-legend>

207. <https://www.youtube.com/watch?v=I3nGsYiBwSs>

208. <https://stackoverflow.com/questions/14406214/moving-x-axis-to-the-top-of-a-plot-in-matplotlib>

209. <https://statisticsglobe.com/move-x-axis-top-plot-r>

210. <https://www.youtube.com/watch?v=X1y0JFKpvZc>

211. https://grapherhelp.goldensoftware.com/Axes/Position_Axis.htm

212. <https://www.thedataschool.co.uk/jevon-dacosta/tableau-tip-switching-x-axis-top-chart/>

213. <https://jakebinstein.com/blog/move-excel-axis-up-down-left-or-right/>

214. https://matplotlib.org/stable/gallery/ticks/ticks_top_right.html

215. <https://www.youtube.com/watch?v=LTbHO92tQI>

216. https://matplotlib.org/stable/gallery/ticks/tick_xlabel_top.html

217. <https://www.stlouisfed.org/on-the-economy/2023/jan/does-greater-economic-prosperity-bring-lower-mortality-rates>

218. <https://paulkrugman.substack.com/p/what-happens-when-americans-realize>

219. <https://www.khanacademy.org/economics-finance-domain/macroeconomics/macro-economic-indicators-and-the-business-cycle/macro-limitations-of-gdp/a/how-well-gdp-measures-the-well-being-of-society-cnx>

220. <https://www.businesstimes.com.sg/opinion-features/there-more-life-and-death-gdp>

221. https://www.reddit.com/r/Anticonsumption/comments/1mela3c/a_perfect_illustration_of_why_gdp_is_a_bad/

222. <https://www.tutorchase.com/answers/ib/politics/what-is-the-correlation-between-life-expectancy-and-economic-growth>

223. <https://www.hinrichfoundation.com/research/infographic/trade-geopolitics/connecting-trade-and-life-expectancy>

224. <https://steadystate.org/guess-what-has-decoupled-from-economic-growth/>

225. <https://www.youtube.com/watch?v=aJ99dXXROvU>

226. <https://pmc.ncbi.nlm.nih.gov/articles/PMC11171643/>

227. <https://www.iza.org/publications/dp/10015/the-intimate-link-between-income-levels-and-life-expectancy-global-evidence-from-213-years>

228. <https://lefteast.org/background-geo-economic-lineup-of-russia-ukraine-east-central-europe/>

229. <https://pubmed.ncbi.nlm.nih.gov/9508159/>

230. <https://pmc.ncbi.nlm.nih.gov/articles/PMC6506569/>

231. <https://hdr.undp.org/system/files/documents/hdrp201032.pdf>

232. <https://data.who.int/countries/643>

233. <https://acikerisim.gelisim.edu.tr/server/api/core/bitstreams/a3102a66-dbf4-4da0-88d2-ac70049a8037/content>

234. https://www.tse-fr.eu/sites/default/files/TSE/documents/doc/wp/2023/wp_tse_1474.pdf

235. <https://ncmu.hse.ru/en/programme-2-2-2>

236. <https://frdelpino.es/investigacion/wp-content/uploads/2022/12/LPE-Health income and the Preston Curve-EHB 2022-1.pdf>

237. https://en.wikipedia.org/wiki/Health_in_Russia

238. image.jpg

239. <https://conversableeconomist.com/2025/04/17/russia-an-unhealthy-population/>

240. https://en.wikipedia.org/wiki/Preston_curve

241. https://web.archive.org/web/20180701002031/http://cliodynamics.ru/index.php?option=com_content&task=view&id=376&Itemid=1

242. <https://docs.iza.org/dp10015.pdf>

243. https://ggplot2.tidyverse.org/reference/aes_colour_fill_alpha.html

244. https://sape.inf.usi.ch/quick-reference/ggplot2/geom_text.html

245. https://ggplot2.tidyverse.org/reference/pattern_alpha.html

246. https://www.rdocumentation.org/packages/plothelper/versions/0.1.9/topics/annotation_transparent_text

247. <https://stackoverflow.com/questions/4001316/how-do-i-preserve-transparency-in-ggplot2>

248. https://ggplot2.tidyverse.org/reference/scale_alpha.html

249. <https://stackoverflow.com/questions/54241972/transparency-in-x-and-y-labels-ggplot2-r>

250. <https://stackoverflow.com/questions/32423167/ggplot-xy-scatter-how-to-change-alpha-transparency-for-select-points>

251. <https://gist.github.com/michaeldorman/48d8e3ad518891057a190aa3149c1e29>

252. <https://stackoverflow.com/questions/59145206/in-ggplot2-how-can-i-make-the-plotted-data-transparent>

253. <https://stackoverflow.com/questions/74969652/adjust-background-alpha-of-geom-richtext>

254. <https://stackoverflow.com/questions/79667147/how-to-set-alpha-transparency-for-a-geom-table-tables-text-and-or-background>

255. https://ggplot2.tidyverse.org/reference/fill_alpha.html

256. https://blog.csdn.net/m0_74331272/article/details/136518479

257. https://search.r-project.org/CRAN/refmans/plothelper/html/annotation_transparent_text.html

258. <https://ggplot2.tidyverse.org/reference/labs.html>

259. <https://databank.worldbank.org/metadataglossary/jobs/series/NY.GDP.PCAP.PP.KD>

260. <http://r-statistics.co/ggplot2-Labels-and-Annotations.html>

261. <https://gist.github.com/Paigehsu/16f354072f795abc61149d56261293dd>

262. <https://search.r-project.org/CRAN/refmans/animint2/html/labs.html>

263. <https://viz-ggplot2.rsquaredacademy.com/ggplot2-labels>

264. <https://www.datanovia.com/en/blog/how-to-change-ggplot-labels/>

265. <https://info3312.infosci.cornell.edu/ae/ae-04-scales-guides-A.html>

266. https://comet.arts.ubc.ca/docs/Beginner/beginner_intro_to_data_visualization2/beginner_intro_to_data_visualization2.html

267. <https://r4ds.hadley.nz/communication.html>

268. <https://www.r-bloggers.com/2022/11/exercises-in-plotting-wdi-data/>

